

FEDERAL UNIVERSITY OF EDUCATION, KONTAGORA
DEPARTMENT OF COMPUTER SCIENCE

CSC 102: PROBLEM SOLVING AND ALGORITHMS

Credit Unit: 3

COURSE CONTENT

MODULE 1: INTRODUCTION TO COMPUTING AND PROBLEM SOLVING

MODULE 2: TYPES OF PROBLEMS

MODULE 3: ALGORITHMS AND HEURISTICS

MODULE 4: PROBLEM-SOLVING TECHNIQUES

MODULE 5: ALGORITHM DESIGN AND DEVELOPMENT

MODULE 6: PSEUDOCODE

MODULE 7: FLOWCHARTS

MODULE 8: DECISION-MAKING STRUCTURES

MODULE 1: INTRODUCTION TO COMPUTING AND PROBLEM SOLVING

1.0 INTRODUCTION

The rapid advancement of Information and Communication Technology (ICT) has transformed virtually every aspect of human life. Modern organizations rely on computer systems for efficient management of information, decision-making, communication, and service delivery.

At the center of all computing activities lies the concept of problem solving. Every software application, database system, mobile application, website, and artificial intelligence system is developed to solve one or more problems.

For example:

- Banking software solves problems related to financial transactions.
- Hospital management systems solve problems related to patient record management.
- School portals solve problems related to student registration and result processing.
- Navigation systems solve route-finding problems.

Before any computer system can be developed, the problem to be solved must first be understood and analyzed. Therefore, problem solving is regarded as the foundation of Computer Science and software development.

This module introduces students to the concepts of computing, problems, and problem solving, and establishes the relationship between them.

2.0 LEARNING OBJECTIVES

- At the end of this module, students should be able to:
- Define computing.
- Explain the basic components of a computing system.
- Define a problem.
- Identify different forms of problems.
- Define problem solving.
- Explain the importance of problem solving in Computer Science.
- Explain the relationship between computing and problem solving.
- Identify the components of a problem.
- Apply the problem-solving process to real-life situations.

3.0 CONCEPT OF COMPUTING

3.1 Definition of Computing

Computing refers to the use of computers, computational methods, and information systems to process data and solve problems.

Computing is not limited to programming alone. It encompasses the study, design, development, implementation, and management of computer-based systems.

Computing involves:

- Data processing
- Information management
- Computer programming
- Software engineering
- Networking
- Artificial intelligence
- Cybersecurity
- Database management

The primary aim of computing is to transform raw data into useful information that can support decision-making and problem solving.

3.2 Characteristics of Computing

Computing is characterized by:

- **Speed:** Computers perform calculations extremely fast.
- **Accuracy:** Computers produce accurate results when properly programmed.
- **Automation:** Computers can perform tasks automatically once instructions are provided.
- **Reliability:** Computers can repeat tasks consistently without fatigue.
- **Storage Capacity:** Large amounts of information can be stored and retrieved.
- **Versatility:** Computers can perform different tasks in different fields.

4.0 COMPONENTS OF A COMPUTING SYSTEM

A computing system consists of five major components.

4.1 Hardware: Hardware refers to the physical components of a computer system.

Examples: Keyboard, Mouse, Monitor, Printer, CPU, Scanner,

4.2 Software: Software refers to programs and instructions that direct computer operations.

Types of software include:

System Software: Windows, Linux, macOS

Application Software: Microsoft Word, Excel, Web browsers

4.3 Data

Data refers to raw facts and figures that have not yet been processed into meaningful information.

Examples: Student scores, Registration numbers, Salary figures

4.4 People (Humanware)

Human beings are responsible for operating and managing computer systems.

Examples: Programmers, System analysts, Database administrators, End users

4.5 Procedures

Procedures are rules and guidelines governing system operations.

Examples: Login procedures, Backup procedures, Examination registration procedures

5.0 CONCEPT OF A PROBLEM

5.1 Meaning of a Problem

A problem is a situation in which there exists a difference between the current condition and the desired condition.

A problem occurs whenever an objective cannot be achieved immediately due to obstacles or constraints.

Example:

Current State: Manual calculation of examination results.

Desired State: Automatic computation of examination results.

Problem: How can result computation be automated?

5.2 Elements of a Problem

Every problem contains the following elements:

- Initial State
- The present condition.
- Goal State
- The desired condition.
- Obstacles
- Factors preventing attainment of the goal.
- Resources
- Available means for solving the problem.

6.0 PROBLEM SOLVING

6.1 Definition of Problem Solving

Problem solving is the systematic process of identifying, analyzing, and resolving a problem in order to achieve a desired goal.

Problem solving involves:

- Understanding the problem
- Analyzing available information
- Designing solutions
- Selecting the best solution
- Implementing the solution
- Evaluating the result

6.2 Importance of Problem Solving

- **Foundation of Programming:** Every computer program is developed to solve a specific problem.
- **Development of Logical Thinking:** Problem solving improves reasoning ability.
- **Better Decision Making:** It enables selection of appropriate alternatives.
- **Innovation and Creativity:** New technologies emerge through problem solving.
- **Increased Productivity:** Problems are solved more efficiently.
- **Economic Development:** Technological solutions contribute to national development.

7.0 RELATIONSHIP BETWEEN COMPUTING AND PROBLEM SOLVING

Computing exists primarily to solve problems.

The relationship can be represented as:

Problem → Analysis → Algorithm → Program → Output → Solution

A computer program is simply a set of instructions developed to solve a problem. Without problems, there would be little need for computer applications.

Practical Examples

- Banking
 - o Problem: Slow transaction processing.
 - o Computing Solution: Computerized banking software.
- Education
 - o Problem: Manual result computation.
 - o Computing Solution: Result processing system.
- Healthcare
 - o Problem: Difficulty managing patient records.
 - o Computing Solution: Electronic Health Record System.

8.0 COMPONENTS OF A COMPUTATIONAL PROBLEM

A computational problem generally consists of:

- Input: Data supplied to the system. Example: Students' scores.
- Process: Operations performed on the data. Example: Addition and averaging.
- Output: Result produced by the system. Example: Average score.

This model is known as the Input–Process–Output (IPO) Model.

Example

Problem: Calculate the average of three scores.

- o Input: Score1, Score2, Score3
- o Process: Add scores and divide by three.
- o Output: Average score.

9.0 GENERAL PROBLEM-SOLVING PROCESS

The problem-solving process is a sequence of logical and systematic steps followed in identifying, analyzing, solving, and evaluating a problem.

The process ensures that solutions are:

- Correct
- Efficient
- Reliable
- Maintainable

In Computer Science, this process often leads to the development of algorithms and computer programs.

The problem-solving process is important because it:

- 1 Provides Direction: It guides problem solvers through organized steps.
- 2 Reduces Errors: Careful analysis helps identify mistakes before implementation.
- 3 Improves Efficiency: Resources are utilized effectively.
- 4 Supports Software Development: Most software development methodologies are based on problem-solving principles.
- 5 Enhances Decision Making: It promotes logical and evidence-based decisions.

Stages of the Problem-Solving Process

The general problem-solving process consists of seven major stages:

1. Problem Identification
2. Problem Definition
3. Problem Analysis
4. Solution Design
5. Solution Implementation
6. Testing and Evaluation
7. Maintenance and Improvement

Stage 1: Problem Identification: Problem identification is the process of recognizing that a problem exists. This is the first and most important stage because a problem cannot be solved unless it is first identified.

Activities Involved

- Recognizing symptoms.
- Observing situations.

- Collecting preliminary information.
- Determining whether a problem truly exists.

Example

A university notices that students spend several days registering courses manually.

This observation indicates a potential problem.

Questions to Ask: What is happening? Why is it happening? Who is affected? How serious is the problem?

Stage 2: Problem Definition: Problem definition involves clearly stating the problem and establishing its boundaries. A poorly defined problem often leads to poor solutions.

Characteristics of a Good Problem Definition

A good problem definition should be:

- Clear
- Specific
- Measurable
- Relevant
- Achievable

Example

Poor Definition: "Registration is difficult."

Good Definition: "Students experience delays of more than three days during course registration due to manual processing."

Importance

A clear definition ensures that everyone understands the problem in the same way.

Stage 3: Problem Analysis

Problem analysis involves studying the problem carefully to understand its causes, requirements, and constraints.

Activities

- Gathering Information
- Collect relevant data about the problem.
- Identifying Inputs
- Determine the data required.
- Identifying Outputs
- Determine expected results.
- Identifying Constraints
- Determine limitations.

Examples:

Time constraints

Financial constraints

Technical constraints

Example

For an online registration system:

Inputs: Student details, Course information

Outputs: Registration slip

Constraints: Internet connectivity, Server capacity

Stage 4: Solution Design: Solution design involves developing one or more possible solutions to the problem.

Activities

- Generating Alternatives
- Develop different approaches.
- Evaluating Alternatives
- Assess advantages and disadvantages.
- Selecting the Best Solution
- Choose the most suitable alternative.

Tools Used: Algorithms, Flowcharts, Pseudocode, Models

Example

Possible solutions to registration delays:

1. Increase staff.
2. Introduce online registration.
3. Extend registration period.

The best option may be online registration.

Stage 5: Solution Implementation: Implementation involves putting the chosen solution into operation.

Activities

- Developing the system.
- Installing hardware.
- Writing programs.
- Training users.
- Deploying solutions.

Example

Developing and launching an online course registration portal.

Challenges: User resistance, Technical difficulties, Financial limitations.

Stage 6: Testing and Evaluation: Testing and evaluation determine whether the implemented solution solves the problem effectively.

Objectives

Verify correctness.

Identify errors.

Assess performance.

Types of Testing

- Functional Testing
- Checks whether the solution performs expected functions.
- Performance Testing
- Measures speed and efficiency.
- User Acceptance Testing
- Ensures users are satisfied.

Example

Testing an online registration system by allowing students to register and checking whether registration records are generated correctly.

Stage 7: Maintenance and Improvement

Maintenance involves modifying and improving a solution after implementation.

Reasons for Maintenance

- Corrective Maintenance
- Fixing errors.
- Adaptive Maintenance
- Adjusting to new requirements.
- Perfective Maintenance
- Improving performance.
- Preventive Maintenance
- Preventing future problems.

Example

Updating a registration portal to accommodate new academic policies.

The Input–Process–Output (IPO) Model

The IPO model is a simple framework used during problem analysis.

Input: Data supplied to the system.
Process: Operations performed on the data.
Output: Results produced.

Example

Problem: Calculate a student's average score.

Input: Three scores

Process: Add scores and divide by three

Output: Average score

The averaging operation is represented by:

$$\text{Average} = \frac{\text{Score}_1 + \text{Score}_2 + \text{Score}_3}{3}$$

Applying the Problem-Solving Process to Computing

In Computer Science, the problem-solving process is often represented as:

Problem → Analysis → Algorithm → Flowchart/Pseudocode → Program → Testing → Solution

Each stage contributes to the development of a reliable software system.

Common Mistakes in Problem Solving

1. Poor problem definition.
2. Inadequate analysis.
3. Ignoring constraints.
4. Selecting unsuitable solutions.
5. Insufficient testing.
6. Lack of maintenance planning.

Benefits of a Structured Problem-Solving Process

- Improved accuracy.
- Better decision making.
- Increased efficiency.
- Reduced risk.
- Easier maintenance.
- Higher user satisfaction.

Summary

The problem-solving process provides a systematic framework for identifying, analyzing, designing, implementing, testing, and maintaining solutions. Following these stages ensures that problems are solved efficiently and effectively. In computing, the problem-solving process serves as the foundation for algorithm development and software engineering.

10.0 PÓLYA'S MODEL OF PROBLEM SOLVING

The mathematician George Pólya proposed a four-stage model.

Step 1: Understand the Problem: Determine what is known and what is required.

Step 2: Devise a Plan: Develop a strategy for solving the problem.

Step 3: Carry Out the Plan: Implement the strategy.

Step 4: Review the Solution: Evaluate and improve the solution.

This model remains one of the most influential approaches to problem solving.

BARRIERS TO EFFECTIVE PROBLEM SOLVING

Common barriers include:

- Poor problem definition.
- Lack of information.
- Emotional bias.
- Inadequate resources.
- Fear of failure.
- Lack of creativity.
- Resistance to change.

12.0 CASE STUDY

Student Result Processing System

Problem

A lecturer spends several days computing examination results manually.

Analysis

Input:

Continuous assessment scores

Examination scores

Process:

Calculate total score

Determine grades

Output:

Student grades

Class statistics

Solution

Develop a computerized result-processing system.

Benefits

- Accuracy
- Speed
- Reliability
- Reduced workload

SUMMARY

Computing is the use of computer systems and computational techniques to process data and solve problems. Problem solving is a systematic process of identifying and resolving challenges. The success of computing depends largely on effective problem-solving skills. Understanding problems, analyzing requirements, and designing solutions are essential competencies for every Computer Science student.

MODULE 2: TYPES OF PROBLEMS

1.0 Introduction

Before a problem can be solved effectively, it must first be understood and classified. Problems differ in their nature, complexity, structure, and methods of solution. Some problems can be solved using established procedures, while others require creativity, critical thinking, and extensive analysis.

In Computer Science, understanding the type of problem being addressed helps in selecting the appropriate tools, techniques, algorithms, and resources needed for solving it. Proper problem classification also assists programmers and system analysts in designing efficient solutions.

This module introduces students to the various types of problems commonly encountered in computing and everyday life.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Explain the concept of problem classification.
2. Identify different types of problems.
3. Differentiate between routine and non-routine problems.
4. Explain solvable and unsolvable problems.
5. Describe computational problems.
6. Distinguish between well-defined and ill-defined problems.
7. Apply problem classification to real-world situations.

3.0 Meaning of Problem Classification

Problem classification is the process of grouping problems according to their characteristics, complexity, structure, and methods of solution.

Problem classification helps problem solvers to:

- Understand problems better.
- Select appropriate solution techniques.
- Improve efficiency.
- Save time and resources.
- Develop suitable algorithms.

4.0 Major Types of Problems

Problems can be classified into the following categories:

1. Routine Problems
2. Non-Routine Problems
3. Well-Defined Problems
4. Ill-Defined Problems
5. Solvable Problems
6. Unsolvable Problems
7. Computational Problems
8. Real-World Problems

5.0 Routine Problems

Routine problems are problems that can be solved using established rules, formulas, procedures, or algorithms. Their solutions are usually straightforward because the method of solution is already known.

Characteristics

- Clearly defined.
- Standard procedures exist.
- Predictable outcomes.
- Usually have one correct answer.
- Require little creativity.

Examples 1. Calculating simple interest. 2. Finding the area of a rectangle. 3. Computing students' GPA. 4. Payroll calculations.

5. Currency conversion.

Example

A student is asked to calculate the average of three scores.

The procedure is already known; therefore, it is a routine problem.

Advantages

- Easy to solve.
- Less time-consuming.
- Suitable for automation.

6.0 Non-Routine Problems

Non-routine problems are problems for which no direct formula or standard procedure exists. They require creativity, reasoning, experimentation, and innovation.

Characteristics

- Complex in nature.
- Multiple solution paths.
- Require critical thinking.
- May have several acceptable solutions.

Examples

1. Designing a smart city.
2. Developing an AI-powered teaching assistant.
3. Reducing traffic congestion.
4. Creating a national cybersecurity framework.

Example: A university wants to use artificial intelligence to predict students at risk of failing examinations.
No ready-made procedure exists.
Different approaches must be explored.
This is a non-routine problem.

7.0 Well-Defined Problems

A well-defined problem is one in which the objectives, inputs, outputs, and constraints are clearly specified.

Characteristics

- Clear goals.
- Clear inputs.
- Clear outputs.
- Easy evaluation criteria.

Example

Problem: Calculate the total score of a student.

Inputs: Test score, Examination score

Output: Total score

Since all requirements are clear, this is a well-defined problem.

8.0 Ill-Defined Problems

Ill-defined problems lack clearly specified goals, inputs, outputs, or constraints.

Characteristics

- Ambiguous requirements.
- Uncertain solutions.
- Multiple interpretations.
- Difficult evaluation criteria.

Examples

1. Improve educational quality.
2. Reduce corruption.
3. Improve national security.
4. Increase student satisfaction.

Example: "Improve learning in Nigerian universities."
Different people may interpret this problem differently.

Therefore, it is an ill-defined problem.

9.0 Solvable Problems

Solvable problems are problems for which a practical solution can be obtained using available knowledge, methods, and resources.

Examples

1. Student registration systems.
2. Online payment systems.
3. Result processing systems.
4. Library management systems.

Characteristics

- Solution exists.
- Resources are available.
- Can be implemented successfully.

10.0 Unsolvable Problems

Unsolvable problems are problems that cannot be solved under given conditions or for which no solution currently exists.

Examples

1. Predicting every future event with complete accuracy.
2. Certain theoretical computing problems.
3. Constructing a perpetual-motion machine.

Note

Some problems that appear unsolvable today may become solvable with future technological advancements.

11.0 Computational Problems

Computational problems are problems that can be solved using computers and algorithms.

These are the primary focus of Computer Science.

Characteristics

- Require data processing.
- Can be represented logically.
- Can be automated.

Examples

1. Searching for a student's record.
2. Sorting examination scores.
3. Calculating payroll.
4. Processing online registrations.
5. Route optimization.

Computational Problem Model

Every computational problem consists of: **Input** → **Process** → **Output**

Example:

Input: Student scores

Process: Calculate average

Output: Average score

12.0 Real-World Problems

Real-world problems are practical challenges encountered in everyday life, organizations, industries, and society.

Examples: Poverty, Unemployment, Traffic congestion, Environmental pollution, Disease outbreaks

Computer scientists often develop technological solutions to address these challenges.

13.0 Factors Affecting Problem Complexity

The complexity of a problem depends on:

- **Size of the Problem:** Larger problems are generally more difficult.
- **Amount of Information:** Insufficient information increases difficulty.
- **Number of Variables:** More variables increase complexity.
- **Constraints:** Restrictions make problems harder to solve.
- **Available Resources:** Time, money, manpower, and technology influence problem-solving efforts.

14.0 Importance of Problem Classification

Problem classification helps to:

1. Improve understanding of problems.
2. Select appropriate solution methods.
3. Design suitable algorithms.
4. Save time and resources.
5. Improve decision-making.
6. Increase efficiency.

15.0 Comparison of Problem Types

Routine Problems

Standard procedures exist
Usually simple
One correct answer
Less creativity required

Non-Routine Problems

No standard procedures
Usually complex
Multiple possible answers
More creativity required

Well-Defined Problems

Clear objectives
Known inputs and outputs
Easy evaluation

Ill-Defined Problems

Ambiguous objectives
Unclear inputs and outputs
Difficult evaluation

16.0 Case Study

Development of an Online Hostel Allocation System

Problem: Manual hostel allocation is slow and prone to errors.

Classification: Computational Problem, Solvable Problem, Well-Defined Problem

Inputs: Student details, Hostel capacity, Allocation rules

Process: Verify eligibility: Assign rooms

Output: Hostel allocation list

Solution: Develop a computerized hostel allocation system.

17.0 Summary

Problems vary in complexity, structure, and methods of solution. Understanding problem types helps Computer Science students select suitable techniques and develop effective solutions. Problems may be routine, non-routine, well-defined, ill-defined, solvable, unsolvable, computational, or real-world in nature.

MODULE 3: ALGORITHMS AND HEURISTICS

1.0 Introduction

After identifying and analyzing a problem, the next step in problem solving is to develop a solution procedure. In Computer Science, this solution procedure is known as an algorithm.

Algorithms are the foundation of all computer programs. Every application software, website, mobile application, operating system, and artificial intelligence system operates based on one or more algorithms.

However, not all problems can be solved immediately using a precise algorithm. In many situations, problem solvers employ heuristics, which are practical approaches or "rules of thumb" used to find satisfactory solutions when exact methods are difficult or unavailable.

This module introduces the concepts of algorithms and heuristics and explains their role in problem solving and software development.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Define an algorithm.
2. Explain the importance of algorithms.
3. State the characteristics of a good algorithm.
4. Differentiate between algorithms and heuristics.
5. Explain heuristic approaches to problem solving.
6. Develop simple algorithms.
7. Apply computational thinking in solving problems.

3.0 Meaning of an Algorithm

An algorithm is a finite sequence of well-defined instructions designed to solve a problem or perform a specific task. In simple terms, an algorithm is a step-by-step procedure for obtaining a solution to a problem.

An algorithm can also be described as:

- A systematic procedure for solving a problem.
- A logical sequence of instructions.
- A blueprint for computer programming.

Before writing a computer program, a programmer usually develops an algorithm.

5.0 Importance of Algorithms

Algorithms are important because they:

- 1 Provide Clear Solutions: They present a systematic approach to solving problems.
- 2 Facilitate Programming: Programs are easier to write when algorithms are developed first.
- 3 Improve Accuracy: Algorithms reduce errors by providing clear instructions.
- 4 Save Time: A well-designed algorithm simplifies implementation.
- 5 Promote Reusability: The same algorithm can be applied to similar problems.
- 6 Improve Efficiency: Efficient algorithms reduce processing time and resource usage.

6.0 Characteristics of a Good Algorithm

A good algorithm should possess the following characteristics.

- 1 Input: An algorithm should accept zero or more inputs.
- 2 Output: An algorithm should produce at least one output.
- 3 Definiteness: Every instruction must be clear and unambiguous.
- 4 Finiteness: The algorithm must terminate after a finite number of steps.
- 5 Effectiveness: Each instruction must be practical and executable.
- 6 Generality: The algorithm should solve a class of similar problems, not just one specific case.

7.0 Structure of an Algorithm

Most algorithms consist of three major parts:

Input: Data supplied to the algorithm.

Processing: Operations performed on the data.

Output: Results produced by the algorithm.

This is known as the Input–Process–Output (IPO) model.

8.0 Everyday Examples of Algorithms

Algorithms are not limited to computers.

Example 1: Making Tea

1. Boil water.
2. Put tea bag in cup.
3. Pour hot water into cup.
4. Add sugar.
5. Stir.
6. Serve.

Example 2: ATM Cash Withdrawal

1. Insert ATM card.
2. Enter PIN.
3. Select withdrawal.
4. Enter amount.
5. Confirm transaction.
6. Collect cash.
7. Remove card.

Example 3: Logging into a School Portal

1. Open portal.
2. Enter username.
3. Enter password.
4. Click login.
5. Access dashboard.

9.0 Writing Algorithms

Algorithms can be written in several forms:

1. Plain English
2. Pseudocode
3. Flowcharts
4. Programming Languages

At this stage, plain English algorithms are sufficient.

10.0 Example Algorithms

Example 1: Addition of Two Numbers

Problem: Add two numbers.

Algorithm:

1. Start
2. Input A and B
3. $\text{Sum} = A + B$
4. Display Sum
5. Stop

Example 2: Calculate Average of Three Scores

Problem: Find the average score of three students.

The computation is:

$$\text{Average} = \frac{\text{Score}_1 + \text{Score}_2 + \text{Score}_3}{3}$$

Algorithm:

1. Start
2. Input Score1, Score2, Score3
3. Calculate Total
4. Calculate Average
5. Display Average
6. Stop

11.0 What is a Heuristic?

A heuristic is a practical approach used to find a satisfactory solution when an exact solution is difficult, expensive, or time-consuming.

A heuristic is often called a rule of thumb.

Unlike algorithms, heuristics do not always guarantee the best solution.

12.0 Characteristics of Heuristics

1. Practical in nature.
2. Faster than exhaustive methods.
3. May not guarantee optimal solutions.
4. Useful in complex situations.
5. Depend on experience and judgment.

13.0 Examples of Heuristics

- Trial and Error: Trying different possibilities until a solution is found.
Example: Trying several passwords until the correct one is discovered.
- Guess and Check: Making an intelligent guess and verifying the result.
Example: Estimating the answer to a mathematical problem.
- Working Backward: Starting from the desired goal and tracing backward.
Example: Solving puzzles.
- Means-End Analysis: Comparing the current state with the desired state and reducing the difference.
Example: Planning study schedules to improve academic performance.

14.0 Difference Between Algorithms and Heuristics

Algorithm	Heuristic
Exact procedure	Approximate procedure
Produces correct result	May produce satisfactory result
Well-defined steps	Flexible approach
Guaranteed solution	No guarantee of optimal solution
Used in programming	Used in decision-making and complex problem solving

15.0 Computational Thinking

Computational thinking is a problem-solving approach that involves formulating problems in a way that allows computers to assist in solving them.

It is a fundamental skill for Computer Science students.

16.0 Components of Computational Thinking

- 1 Decomposition: Breaking a large problem into smaller parts.
Example: Developing a school management system by dividing it into modules.
- 2 Pattern Recognition: Identifying similarities among problems.
Example: Recognizing that payroll and result-processing systems both involve calculations.
- 3 Abstraction: Focusing on important information while ignoring unnecessary details.
Example: Representing a student using only relevant attributes such as matriculation number and scores.
- 4 Algorithm Design: Developing step-by-step procedures for solving problems.
Example: Creating instructions for computing GPA.

17.0 Advantages of Computational Thinking

1. Improves logical reasoning.
2. Enhances analytical skills.
3. Supports software development.
4. Encourages systematic problem solving.
5. Facilitates automation.

18.0 Case Study

Student Result Processing

Problem: Calculate students' final grades.

Computational Thinking Approach

Decomposition:

- Break the task into:
- Input scores
- Calculate totals

Determine grades
Print results

Pattern Recognition
All students follow the same grading procedure.

Abstraction
Focus only on relevant academic records.

Algorithm Design
Develop step-by-step instructions for grade computation.

19.0 Summary

Algorithms are step-by-step procedures used to solve problems and form the foundation of programming. A good algorithm must be definite, finite, effective, and capable of producing outputs from inputs. Heuristics are practical methods used when exact solutions are difficult to obtain. Computational thinking provides a systematic framework for analyzing and solving problems efficiently.

MODULE 4: PROBLEM-SOLVING TECHNIQUES

1.0 Introduction

Problem solving is one of the most important skills in Computer Science and in everyday life. While algorithms provide systematic procedures for solving problems, problem-solving techniques provide methods and strategies that help individuals understand problems, generate ideas, evaluate alternatives, and develop solutions.

Many real-world problems are complex and cannot be solved by a single approach. Therefore, problem solvers often employ different techniques depending on the nature of the problem.

This module introduces major problem-solving techniques commonly used in computing, mathematics, science, engineering, business, and management.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Explain the concept of problem-solving techniques.
2. Describe various problem-solving techniques.
3. Apply decomposition to solve complex problems.
4. Use abstraction in problem analysis.
5. Explain divide-and-conquer strategies.
6. Apply brainstorming techniques.
7. Explain means-end analysis.
8. Discuss root-cause analysis.
9. Select appropriate techniques for different problems.

3.0 Meaning of Problem-Solving Techniques

Problem-solving techniques are systematic approaches, methods, or strategies used to understand, analyze, and solve problems.

These techniques help individuals:

- Understand problems clearly.
- Generate solutions.
- Make informed decisions.
- Improve efficiency.
- Reduce errors.

4.0 Importance of Problem-Solving Techniques

Problem-solving techniques are important because they:

- 1 Improve Understanding:** They help identify the real nature of problems.
- 2 Enhance Creativity:** They encourage generation of innovative solutions.
- 3 Save Time:** They provide structured approaches to problem solving.
- 4 Improve Decision Making:** They help select the most suitable solution.
- 5 Reduce Complexity:** Large problems become easier to manage.

Some of the Problem Solving Techniques are:

i. Decomposition: Decomposition is the process of breaking a large and complex problem into smaller and more manageable sub-problems.

Each sub-problem can then be solved independently.

Why Decomposition is Important

- Large problems are often difficult to understand.
- Breaking them into smaller parts makes them:
 - Easier to analyze.
 - Easier to implement.
 - Easier to test.
 - Easier to maintain.

Example

Problem: Develop a Student Management System.

Decomposition

The system can be divided into:

1. Student Registration Module
2. Course Registration Module
3. Result Processing Module

- 4. Hostel Allocation Module
- 5. Payment Module

Each module can be developed separately.

Advantages

- Simplifies problem solving.
- Encourages teamwork.
- Reduces development time.

ii. Abstraction

Abstraction is the process of focusing on important information while ignoring unnecessary details. It helps simplify complex systems.

Example

When designing a student database, important information may include:

Matriculation Number, Name, Department, Level

Details such as favorite food or shoe size may be irrelevant.

Importance

- Reduces complexity.
- Improves understanding.
- Supports efficient system design.

iii. Pattern Recognition

Pattern recognition involves identifying similarities or recurring features among problems. By recognizing patterns, solutions from previous problems can be adapted to new situations.

Example 1

Computing students' GPA and computing employees' salaries both involve:

Input

Processing

Output

Example 2

Many e-commerce websites follow similar structures:

Product selection

Shopping cart

Payment system

Benefits

- Saves time.
- Encourages reuse of solutions.
- Improves efficiency.

iv. Divide-and-Conquer Technique

Divide-and-conquer is a problem-solving strategy that divides a problem into smaller independent parts, solves each part separately, and combines the results.

Steps

1. Divide the problem.
2. Solve each sub-problem.
3. Combine solutions.

Example

Searching for a name in a sorted list.

Instead of checking every name:

Divide the list into two halves.

Determine which half contains the target.

Repeat until the target is found.

Applications

- Searching algorithms
- Sorting algorithms
- Database systems
- Artificial Intelligence

v. Brainstorming

Brainstorming is a technique used to generate multiple ideas for solving a problem. Participants freely suggest ideas without criticism.

Rules of Brainstorming

1. Encourage all ideas.
2. Avoid criticism.
3. Focus on quantity.
4. Build upon others' ideas.

Example

Problem: Reduce examination malpractice.

Possible ideas:

Computer-based testing
Biometric verification
CCTV surveillance
Random seating arrangements

Advantages

Encourages creativity.
Generates many solutions.
Promotes teamwork.

vi. Trial-and-Error Technique

Trial-and-error involves attempting different solutions until the correct one is found.

Example

A student forgets a password and tries different possibilities.

Advantages

Simple to apply.
Useful when little information is available.

Disadvantages

Time-consuming.
May be inefficient.

vii. Means-End Analysis

Means-end analysis is a technique that compares the current situation with the desired situation and identifies actions needed to reduce the difference.

Components

- Current State: Present condition.
- Goal State: Desired condition.
- Means: Actions required to achieve the goal.

Example

Current State: Student's GPA = 2.00

Desired State: Student's GPA = 4.00

Means:

Attend lectures regularly.
Study consistently.
Complete assignments.

Advantages

- Goal-oriented.
- Encourages planning.

viii. Root-Cause Analysis

Root-cause analysis seeks to identify the underlying cause of a problem rather than merely addressing symptoms.

Example:

Problem: Students perform poorly in examinations.

Possible Causes:

- Poor study habits
- Inadequate teaching resources
- Poor attendance
- Lack of preparation

The actual cause must be identified before proposing solutions.

Benefits

- Prevents recurrence of problems.
- Produces long-term solutions.

ix. Research Technique

Research involves collecting and analyzing information to understand and solve problems.

Sources of Information

- Books
- Journals
- Internet resources
- Interviews
- Surveys

Example

A software developer researching cybersecurity threats before designing a security system.

x. Hypothesis Testing

A hypothesis is a proposed explanation that can be tested.

Hypothesis testing involves verifying whether the proposed explanation is correct.

Example

Hypothesis:

"Poor internet connectivity causes delays in online registration."

Data is collected and analyzed to determine whether the hypothesis is true.

xi. Analogy Technique

Analogy involves solving a new problem using solutions from similar previous problems.

Example

Developing a university portal based on the design of an existing portal used in another institution.

Benefits

- Saves time.
- Promotes reuse of knowledge.

xii. Morphological Analysis

Morphological analysis involves examining all possible combinations of solution components.

Example

Designing a mobile phone by considering different combinations of:

- Battery types
- Screen sizes
- Operating systems

- Storage capacities

xiii. Literal Thinking

Literal thinking focuses strictly on facts and explicit information provided.

Example

Following examination instructions exactly as written.

Selecting Appropriate Problem-Solving Techniques

The choice of technique depends on:

- Nature of the problem
- Complexity level
- Available resources
- Time constraints
- Desired accuracy

Summary

Problem-solving techniques provide structured approaches to solving problems. Important techniques include decomposition, abstraction, pattern recognition, divide-and-conquer, brainstorming, trial-and-error, means-end analysis, root-cause analysis, research, hypothesis testing, analogy, and morphological analysis. These techniques help individuals and organizations develop effective and efficient solutions.

Next Module: Module 5 – General Problem-Solving Process, which focuses on problem definition, analysis, solution design, implementation, testing, evaluation, and maintenance.

MODULE 5: ALGORITHM DESIGN AND DEVELOPMENT

1.0 Introduction

One of the most important activities in problem solving is the design of algorithms. After a problem has been identified, analyzed, and understood, the next step is to develop a systematic procedure for solving it.

An algorithm serves as a blueprint for a computer program. Just as an architect prepares a building plan before construction, a programmer designs an algorithm before writing a program.

A well-designed algorithm makes program development easier, improves accuracy, reduces errors, and enhances software quality.

This module introduces students to the principles, methods, and techniques involved in algorithm design and development.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Define algorithm design.
2. Explain the importance of algorithm design.
3. Describe the stages of algorithm development.
4. Explain the principles of good algorithm design.
5. Develop algorithms for simple computational problems.
6. Apply stepwise refinement in algorithm design.
7. Explain modular algorithm design.
8. Verify and evaluate algorithms.

3.0 Meaning of Algorithm Design

Algorithm design is the process of creating a step-by-step procedure for solving a problem.

It involves transforming a problem specification into a logical sequence of instructions that can eventually be implemented as a computer program.

Example

Problem: Calculate the area of a rectangle.

Algorithm:

1. Start
2. Input Length and Width
3. Calculate Area
4. Display Area
5. Stop

The area is computed using:

Area=Length\times Width

Characteristics of Good Algorithm Design

A good algorithm should be:

Correct: It must solve the intended problem accurately.

Efficient: It should use minimal resources.

Simple: The logic should be easy to understand.

Reliable: It should produce consistent results.

Flexible: It should be adaptable to changing requirements.

Maintainable: Future modifications should be easy.

4.0 Importance of Algorithm Design

Algorithm design is important because it:

- 1 Provides a Clear Solution Plan: It enables programmers to understand the logic of a solution before coding.
- 2 Reduces Programming Errors: Logical mistakes can be identified before implementation.
- 3 Improves Efficiency: Well-designed algorithms use fewer resources and less time.
- 4 Simplifies Testing: Errors can be traced more easily.
- 5 Supports Documentation: Algorithms provide a record of system logic.

5.0 Stages of Algorithm Development

Algorithm development involves several stages:

1. Problem Understanding: The first stage involves understanding exactly what needs to be solved. Questions to ask include: What is the problem? What information is available? What result is expected?

2. Problem Analysis: Problem analysis involves identifying:

Inputs: Data supplied to the algorithm.

Example:

Score1

Score2

Score3

Outputs: Expected results.

Example:

Average score

Processing Requirements: Operations to be performed.

Example:

Addition

Division

3. Solution Design

Solution design involves developing logical steps for solving the problem.

Example

Problem: Calculate the average of three scores.

Solution Steps:

1. Receive three scores.
2. Add the scores.
3. Divide the sum by three.
4. Display the result.

The formula used is:

$$\text{Average} = \frac{\text{Score}_1 + \text{Score}_2 + \text{Score}_3}{3}$$

4. Algorithm Representation

5. Verification: Algorithm verification is the process of checking whether an algorithm correctly solves a problem.

6. Optimization: Optimization involves improving an algorithm to make it faster, simpler, or more efficient.

7. Implementation

Summary

Algorithm design is a crucial stage in problem solving and software development. It involves understanding problems, analyzing requirements, developing logical solutions, verifying correctness, and optimizing performance. Techniques such as stepwise refinement, modular design, top-down design, and bottom-up design help programmers create effective and maintainable algorithms.

MODULE 6: PSEUDOCODE

1.0 Introduction

After designing an algorithm, the next step in problem solving is to represent it in a clear, structured, and standardized form before converting it into a programming language. One of the most widely used methods for representing algorithms is pseudocode.

Pseudocode acts as a bridge between human thinking and computer programming. It is not an actual programming language, but it uses structured English-like statements that are easy to understand and easy to convert into code.

In software development, pseudocode helps programmers focus on logic rather than syntax.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Define pseudocode.
2. Explain the importance of pseudocode.
3. Identify rules for writing pseudocode.
4. Write pseudocode for simple problems.
5. Convert algorithms into pseudocode.
6. Understand structured logic using pseudocode.
7. Prepare for programming implementation using pseudocode.

3.0 Meaning of Pseudocode

Pseudocode is an informal, high-level description of an algorithm that uses simple English-like statements combined with programming logic.

It is called “pseudo” because it resembles code but is not executable on a computer.

Simple Definition

Pseudocode is a step-by-step written outline of a program logic in human-readable form.

4.0 Importance of Pseudocode

Pseudocode is important because it:

- 1 Simplifies Programming Logic: It helps programmers focus on solving the problem without worrying about syntax errors.
- 2 Acts as a Blueprint for Programs: It serves as a guide during coding.
- 3 Improves Communication: It can be easily understood by programmers and non-programmers.
- 4 Reduces Errors: Logical errors are identified before actual coding.
- 5 Saves Time: it reduces debugging effort during programming.

5.0 Characteristics of Good Pseudocode

A good pseudocode should be:

- Clear: Statements must be easy to understand.
- Structured: It should follow logical order.
- Language Independent: It should not depend on any programming language.
- Finite: It must end after completing the task.
- Precise: Each instruction must be unambiguous.

6.0 Rules for Writing Pseudocode

1. Use simple English statements.
2. Each instruction should be on a new line.
3. Use keywords like START, END, INPUT, OUTPUT.
4. Indentation should be used for decision and loop structures.
5. Avoid programming language syntax like semicolons.
6. Be consistent in naming variables.
7. Clearly show sequence of steps.

7.0 Basic Pseudocode Structure

A typical pseudocode has:

START, INPUT, PROCESSING, OUTPUT, END

Example: Add two numbers.

Pseudocode:

```

START
INPUT A, B
SUM ← A + B
OUTPUT SUM
END

```

8.0 Pseudocode Statements

1 Input Statement: Used to accept data.

Example: INPUT A, B

2 Output Statement: Used to display results.

Example: OUTPUT SUM

3 Assignment Statement: Used to store values.

Example: SUM ← A + B

9.0 Pseudocode for Simple Problems

Example 1: Addition of Two Numbers

```

START
INPUT A, B
SUM ← A + B
OUTPUT SUM
END

```

Example 2: Check Even or Odd

```

START
INPUT N
IF N MOD 2 = 0 THEN
OUTPUT "Even"
ELSE
OUTPUT "Odd"
END IF
END

```

Example 3: Largest of Two Numbers

```

START
INPUT A, B
IF A > B THEN
OUTPUT A
ELSE
OUTPUT B
END IF
END

```

Example: Calculate average of three numbers.

The formula is:

$$\text{Average} = \frac{A+B+C}{3}$$

Pseudocode:

```

START
INPUT A, B, C
SUM ← A + B + C
AVG ← SUM / 3
OUTPUT AVG
END

```

11.0 Conditional Statements in Pseudocode

IF Statement: Used for decision making.

Example:

```

IF condition THEN
statement

```

END IF

IF...ELSE Statement

```
START
INPUT AGE
IF AGE >= 18 THEN
OUTPUT "Eligible to vote"
ELSE
OUTPUT "Not eligible"
END IF
END
```

Nested IF Statement

```
START
INPUT SCORE
IF SCORE >= 70 THEN
OUTPUT "A"
ELSE IF SCORE >= 60 THEN
OUTPUT "B"
ELSE
OUTPUT "C"
END IF
END
```

Loop Structures in Pseudocode

FOR Loop

```
START
FOR I ← 1 TO 5 DO
OUTPUT I
END FOR
END
```

WHILE Loop

```
START
INPUT N
WHILE N > 0 DO
OUTPUT N
N ← N - 1
END WHILE
END
```

DO WHILE Loop

```
START
DO
INPUT N
WHILE N < 0
END
```

Nested Structures

Nested structures occur when one control structure is placed inside another.

Example:

```
START
FOR I ← 1 TO 3 DO
FOR J ← 1 TO 2 DO
OUTPUT I, J
END FOR
END FOR
END
```

14.0 Advantages of Pseudocode

1. Easy to understand.

2. Helps in planning programs.
3. Independent of programming language.
4. Reduces coding errors.
5. Improves logical thinking.
6. Useful for documentation.

15.0 Limitations of Pseudocode

1. Not standardized globally.
2. Cannot be executed by computers.
3. Different people may write it differently.
4. May be too informal for complex systems.

16.0 Difference Between Pseudocode and Algorithm

Algorithm

Step-by-step procedure
 Can be written in plain English
 Less standardized
 Focus on logic

Pseudocode

Structured representation of algorithm
 Uses structured keywords
 More structured
 Focus on programming readiness

17.0 Real-Life Application

Problem: ATM Withdrawal

Pseudocode:

```

START
INPUT PIN
IF PIN is correct THEN
INPUT AMOUNT
IF BALANCE >= AMOUNT THEN
DISPENSE CASH
ELSE
OUTPUT "Insufficient Funds"
END IF
ELSE
OUTPUT "Invalid PIN"
END IF
END
  
```

18.0 Case Study: Student Result System

```

START
INPUT CA_SCORE, EXAM_SCORE
TOTAL ← CA_SCORE + EXAM_SCORE
IF TOTAL >= 70 THEN
GRADE ← "A"
ELSE IF TOTAL >= 60 THEN
GRADE ← "B"
ELSE IF TOTAL >= 50 THEN
GRADE ← "C"
ELSE
GRADE ← "F"
END IF
OUTPUT TOTAL, GRADE
END
  
```

19.0 Summary

Pseudocode is a structured way of representing algorithms using simple English-like statements. It is an essential step in program development because it helps programmers focus on logic before coding. Pseudocode includes input,

processing, decision-making, and looping structures, making it a powerful tool for designing solutions to computational problems.

MODULE 7: FLOWCHARTS

1.0 Introduction

After developing algorithms and writing pseudocode, the next step in problem solving is to represent the solution visually using diagrams. One of the most important tools for this is the flowchart.

A flowchart provides a graphical representation of the steps involved in solving a problem. It uses standardized symbols connected by arrows to show the flow of logic from start to finish.

Flowcharts are widely used in Computer Science, engineering, business processes, and system design because they make complex logic easier to understand.

2.0 Learning Objectives

At the end of this module, students should be able to:

1. Define a flowchart.
2. Explain the importance of flowcharts.
3. Identify standard flowchart symbols.
4. Draw flowcharts for simple problems.
5. Convert pseudocode into flowcharts.
6. Interpret flowcharts.
7. Understand control flow in problem solving.

3.0 Meaning of Flowchart

A flowchart is a diagrammatic representation of an algorithm or process using symbols to show the sequence of steps. It shows how data moves through a system and how decisions are made.

4.0 Importance of Flowcharts

Flowcharts are important because they:

- 1 Improve Understanding: They make complex logic easier to visualize.
- 2 Aid Communication: They help programmers and non-programmers understand system design.
- 3 Assist Program Development: They serve as a blueprint before coding.
- 4 Reduce Errors: Logical errors can be identified early.
- 5 Support Documentation: They provide permanent system documentation.

5.0 Standard Flowchart Symbols

Flowcharts use standardized symbols:

- 1 Start/End (Terminator):

Oval Shape: Meaning Beginning or end of a process



- 2 Input/Output

Shape: Parallelogram: Meaning Data entry or output display



- 3 Process

Shape: Rectangle: Meaning Calculation or operation



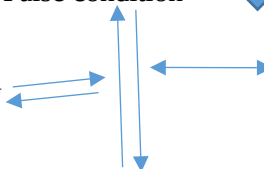
- 4 Decision

Shape: Diamond: Meaning Yes/No or True/False condition



- 5 Flow Lines

Shape: Arrows: Meaning: Direction of flow



- 6 Connector

Shape: Circle: Meaning: Connects parts of flowchart



6.0 Basic Flowchart Structure

A typical flowchart follows:

START → INPUT → PROCESS → DECISION → OUTPUT → END

Rules for Drawing Flowcharts

1. Flow should be top to bottom or left to right.
2. Each flowchart must have a Start and End.

3. Arrows must show correct direction.
4. Avoid crossing lines where possible.
5. Use standard symbols only.
6. Each decision must have Yes/No paths.
7. Keep diagrams simple and clear.

7.0 Flowchart Example 1: Add Two Numbers

Problem: Add two numbers.

Steps

1. Start
2. Input A, B
3. $SUM = A + B$
4. Display SUM
5. End

Flowchart Description

Start (oval)
 Input A, B (parallelogram)
 Process $SUM = A + B$ (rectangle)
 Output SUM (parallelogram)
 End (oval)

8.0 Flowchart Example 2: Even or Odd Number

Problem: Determine if a number is even or odd.

Logic: A number is even if it is divisible by 2.

Flowchart Steps

1. Start
2. Input N
3. Decision: $N \text{ MOD } 2 = 0?$
 Yes → Output "Even"
 No → Output "Odd"
4. End

9.0 Flowchart Example 3: Largest of Two Numbers

Problem: Find the larger number between A and B.

Steps

1. Start
2. Input A, B
3. Decision: $A > B?$
 Yes → Output A
 No → Output B
4. End

10.0 Flowchart Example 4: Average of Three Numbers

The formula is: $\text{Average} = \frac{A+B+C}{3}$

Flowchart Steps

1. Start
2. Input A, B, C
3. $SUM = A + B + C$
4. $AVG = SUM / 3$
5. Output AVG
6. End

15.0 Advantages of Flowcharts

1. Easy to understand visually.
2. Improves communication.
3. Helps in debugging logic.
4. Useful for documentation.
5. Assists program design.

6. Reduces complexity.

16.0 Limitations of Flowcharts

1. Time-consuming to draw.
2. Difficult to modify.
3. Not suitable for very large systems.
4. Can become complex and confusing.
5. Requires drawing tools.

17.0 Difference Between Flowchart and Pseudocode

Flowchart	Pseudocode
Graphical representation	Text-based representation
Uses symbols	Uses structured English
Easier to visualize	Easier to write and modify
Harder to edit	Easier to edit
Space-consuming	Compact

18.0 Real-Life Example: ATM System

Flowchart Logic

1. Start
2. Insert card
3. Enter PIN
4. Is PIN correct?
Yes → Continue
No → Reject card
5. Select transaction
6. Enter amount
7. Check balance
8. Dispense cash
9. End

19.0 Case Study: Student Result System

Flowchart Logic:

1. Start
2. Input CA score
3. Input Exam score
4. $TOTAL = CA + Exam$
5. Is $TOTAL \geq 50$?
Yes → Pass
No → Fail
6. Output result
7. End

20.0 Summary

Flowcharts are graphical tools used to represent algorithms visually. They help in understanding, designing, and documenting problem-solving processes. Flowcharts use standard symbols such as terminators, input/output, process, and decision symbols. They are widely used in software development to simplify complex logic before programming.

11.0 Decision Tables

A decision table is a tabular method used to represent conditions and corresponding actions.

A decision table is a tabular format that lists a set of conditions and the corresponding actions to take for each combination of those conditions. It is especially useful for representing complex conditional ("if-this-then-that") logic compactly and unambiguously.

1.2 Structure

Conditions	Rule 1	Rule 2
Age ≥ 18	Yes	No
Output	Eligible	Not Eligible

11.3 Importance

Organizes complex decisions
Easy to interpret

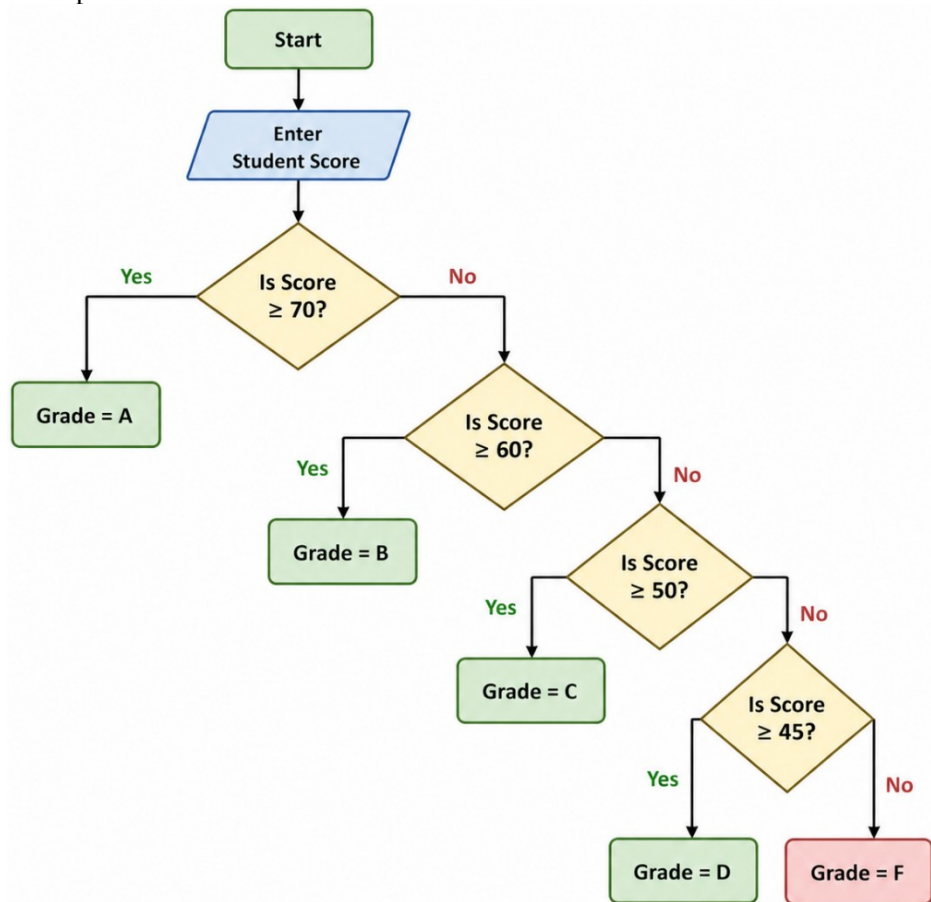
Reduces errors

12.0 Decision Trees

A decision tree is a graphical representation of decision-making processes.

A decision tree* is a tree-like diagram that represents a sequence of decisions and their possible outcomes/consequences, branching outward from a root decision point to leaf outcomes. It is useful for visualizing multi-stage decision logic.

Example



13.0 Applications of Decision-Making

Decision-making structures are used in:

- Education Systems: Grading students, Admission eligibility
- Banking Systems: Loan approval, Transaction verification
- Security Systems: Password authentication
- E-commerce: Discount eligibility

17.0 Summary

Decision-making structures allow programs to make choices based on conditions. They are implemented using IF, IF... ELSE, nested IF, CASE statements, decision tables, and decision trees. These structures make programs intelligent and capable of handling real-world situations.